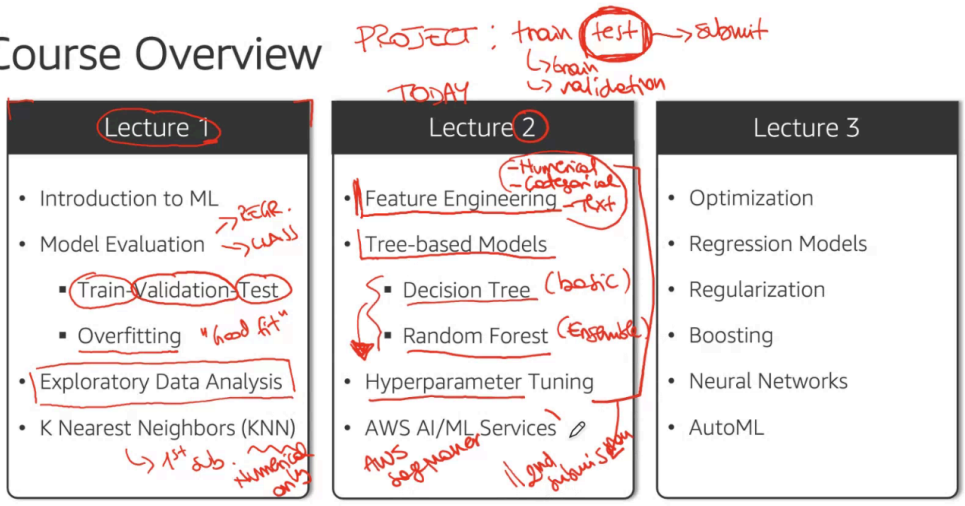
ML Accelerator Tabular Data2

miércoles, 9 de junio de 2021 11:01

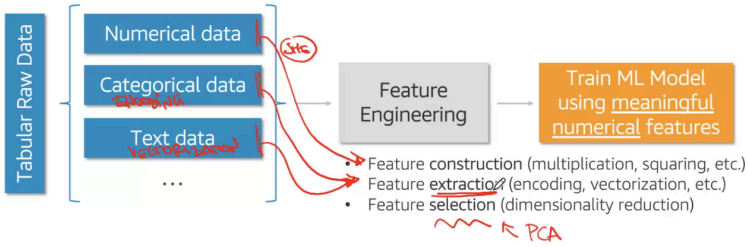
Course Overview



Feature Engineering

Feature Engineering

Intuition: What information would a human expert use to predict?
Often more art than science!



Encoding Categorical Features

Categorical (also called discrete) features: These features don't have a natural numerical representation.

Example: color ∈ {green, red, blue}, isFraud ∈ {false, true}

- Most ML models require converting categorical features to numerical ones.

Encode/define a mapping: Assign a number to each category.

Ordinals: Categories are ordered, e.g., size ∈ {L > M > S}. We can assign L->3, M->2, S->1.

Nominals: Categories are unordered, e.g., color. We can assign the numbers randomly.

Encoding Categorical Features

LabelEncoder: sklearn encoder, encodes target labels with value between 0 and n_classes-1 - .fit(), .transform()

- Encodes target labels values, y (or one feature only!), and not the input X.
- Can be used to transform non-numerical labels or numerical labels.

	color	size	price	classlabel
0	green	S	10.1	shirt
1	red	M	13.5	pants
2	blue	L	15.3	shirt

Let's encode one feature, e.g. the color field.

```
from sklearn.preprocessing import LabelEncoder

le= LabelEncoder()
df['color'] = le.fit_transform(df['color'])
```

	color	size	price	classlabel
0	1	S	10.1	shirt
1	2	M	13.5	pants
2	0	L	15.3	shirt

Encoding Categorical Features

Problem: Encoding categorical features with integers is wrong because the ordering and size of the integers is meaningless.

One-Hot Encoding: Encode a categorical feature into as many new binary features (0s and 1s), as many categories per feature.

OneHotEncoder: sklearn one-hot encoder, encodes categorical features as a one-hot numeric array - .fit(), .transform()

- Does not automatically name the new binary features.
- Works on two or more features (for one-hot encoding of one feature alone should use LabelBinarizer instead!)

get_dummies: pandas one-hot encoder

Encoding Categorical Features

get_dummies: pandas one-hot encoder, converts categorical features into new "dummy"/indicator features.

- Automatically names the new binary features.

```
pd.get_dummies(df, columns=['color'])
```

	color	size	price	classlabel
0	green	S	10.1	shirt
1	red	M	13.5	pants
2	blue	L	15.3	shirt

	size	price	classlabel	color_blue	color_green	color_red
0	S	10.1	shirt	0	1	0
1	M	13.5	pants	0	0	1
2	L	15.3	shirt	1	0	0

Encoding Categorical Features

- Define a hierarchy structure:
  Example: For a zip code feature, can try to use regions -> states -> city as the hierarchy, and can choose a specific level to encode the zip code feature.
- Group/bin the categories into fewer groups by similarity:
  Example: For some user demographics dataset, create age groups: 1-15, 16-22, 23-30, and so forth.

Para evitar la explosión de columnas cuando se lleva a cabo esta técnica, se recomienda definir una jerarquia y situar en el nivel adecuado. Por ejemplo no usar zip y usar ciudad

Encoding Categorical Features

Target Encoding: Encode using values that can explain the target.

Example: Averaging the target value for each category. Then, replace the categorical values with the average target value.

x_1	x_2	y

x_1	x_2	y

a	c	1
a	d	1
b	c	0
a	d	0
a	d	0
a	d	1
b	d	0

x_1 -> cat a -> 3/5 = 0.6
x_1 -> cat b -> 0/2 = 0
x_2 -> cat c -> 1/2 = 0.5
x_2 -> cat d -> 2/5 = 0.4

0.6	0.5	1
0.6	0.4	1
0	0.5	0
0.6	0.4	0
0.6	0.4	0
0.6	0.4	1
0	0.4	0

Cleaning Text Data

- Motivation: Messy text harder to find patterns in.
  - Normalize text by removing noise: convert to lowercases, strip whitespaces, remove special characters, remove markups, etc.

Example: The following two sentences have similar meaning but may seem quite different to a text classifier (i.e. sentiment detector):

- "The countess (Rebecca) considers\n the boy to be quite naïve."
- "countess rebecca considers boy naive"

Tokenization

- Tokenization: Splits text into small parts by white space and punctuation.

Example:

Sentence	Tokens
"I don't like eggs."	"I", "do", "n't", "like", "eggs", "."

Tokens will be used for further cleaning and vectorization.

Stop Words Removal

- Stop Words: Some words that frequently appear in texts, but they don't contribute too much to the overall meaning.
  - Common stop words: "a", "the", "so", "is", "it", "at", "in", "this", "there", "that", "my", "by", "nor"

Example:

Original sentence	Without stop words
"There is a tree near the house."	"tree near house"

Stop Words Removal

Stop Words from the Natural Language Tool Kit (NLTK) library:

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', '
yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it'
s", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that',
"that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having',
'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'a
t', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'bel
ow', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here
', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'suc
h', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "
don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't", 'couldn', "could
n't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", '
ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn',
```

Cuidado que elimina would o wouldnt, así que por ejemplo valoraciones estarían muy impactadas, podría aplicarse a descripciones de articulos

```
wasn't, weren, weren't, won, won't, wouldn, wouldn't ]
```

Is this a good list of stop words for a binary text classification of product reviews (positive or negative review) ?

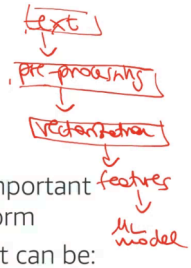
Stemming

- Set of rules to slice a string to a substring that usually refers to a more general meaning.
  - The goal is to remove word affixes (particularly suffixes) such as "s", "es", "ing", "ed", etc.
    - "playing"
    - "played"
    - "plays"

    → "play"
  - The issue: It doesn't usually work with irregular forms such as irregular verbs: **"taught"**, **"brought"**, etc.

Text Vectorization: Bag of Words

ML models need well-defined numerical data.

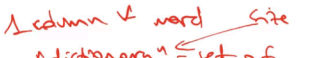


'Bag of Words' (BoW) method:

- Converts **text data** into **numerical** features.
- Referred as feature extraction, as we extracted important information from the original text in a numeric form
- For each word in a document, we get a number; it can be:
  - **binary** (1 or 0, present or not)
  - **word counts or frequencies**

Bag of Words: Binary

Simple example for binary features:



	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	1	0	1	1	1	0	0	0	0
"my cat is old"	0	1	0	1	0	1	0	1	0
"It is not a dog, it is a wolf."	1	0	1	1	1	0	1	0	1

El diccionario se llama a la cantidad de palabras que vamos a modelar con el BoW

Bag of Words: Counts

Simple example for word counts:

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	1	0	1	1	1	0	0	0	0
"my cat is old"	0	1	0	1	0	1	0	1	0
"It is not a dog, it is a wolf."	2	0	1	2	2	0	1	0	1

Bag of Words in sklearn

CountVectorizer: sklearn text vectorizer, converts a collection of text documents to a matrix of token counts - .fit(), .transform()

```
from sklearn.feature_extraction.text import CountVectorizer
countVectorizer = CountVectorizer(binary=True)

sentences = ['This is the first document.',
             'This is the second document.',
             'and the third one.',
            ]

X = countVectorizer.fit_transform(sentences)
print(X.toarray())
```

vocabulary =
{and, document, first, is, one, second, the, third, this}

```
[[0 1 1 1 0 0 1 0 1]
 [0 1 0 1 0 1 1 0 1]
 [1 0 0 0 1 0 1 1 0]]
```

Text Processing Hands-on

- In this notebook we perform the following tasks:
  - Text cleaning
  - Text preprocessing
  - Text vectorization - get binary Bag of Words features

Amazon SageMaker MLA-TAB-DAY2-TEXT-PROCESSING-NB.ipynb

Data Processing with Pipeline (sklearn)

Transformers in sklearn

- SimpleImputer, StandardScaler, MinMaxScaler, LabelEncoder, OrdinalEncoder, OneHotEncoder, and CountVectorizer belong to sklearn's transformers class, all have:
  - .fit() method: learns the transformation from the training dataset
  - .transform() method: applies the transformation to any dataset (training, validation, test) for preprocessing

On training set can also apply .fit_transform()

ColumnTransformer and Pipeline

```
numerical_processing = Pipeline([
    ('num_imputer', SimpleImputer(strategy='mean')),
    ('num_scaler', MinMaxScaler())])
```

y_train

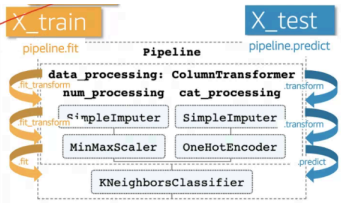
Si aparece un nuevo valor que no estaba en el training que no de error y lo ignore

```
categorical_processing = Pipeline([
    ('cat_imputer', Imputer(strategy='constant', fill_value='missing')),
    ('cat_encoder', OneHotEncoder(handle_unknown='ignore'))])

processor = ColumnTransformer(transformers =[
    ('num_processing', numerical_processing, ('feature1', 'feature3')),
    ('cat_processing', categorical_processing, ('feature0', 'feature2'))])

pipeline = Pipeline([('data_processing', processor),
                     ('estimator', KNeighborsClassifier())])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```



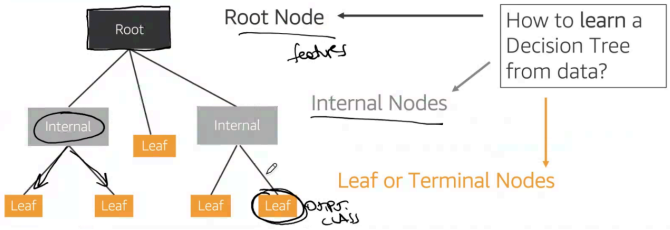
Problemas KNN

No estamos entrenando en realidad, estamos extrapolando.

Si tenemos muchas features, crece demasiado

Decision Trees

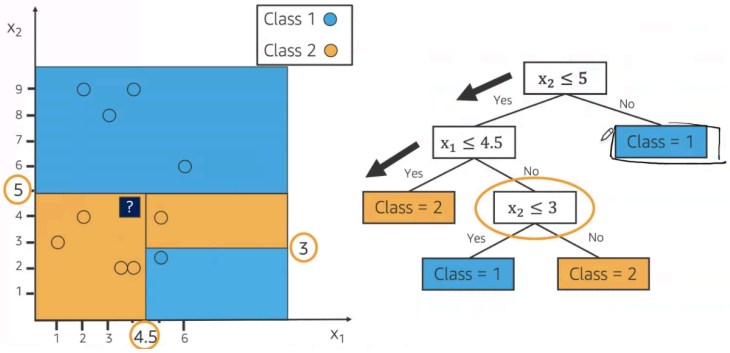
Decision Trees are flowchart-like structures that can be used for classification or regression tasks.



Root is one initial feature, se explica más adelante la mejor primera feature
Each node are features, and leafs are output class

The three created depends on the training dataset.

Decision Trees: Numerical Example



Learn a Decision Tree

ID3* Algorithm:

(Repeat the steps below)

1. Select "the best feature" to split (we will see how to select)
2. Separate the training samples according to the selected feature
3. Stop if we have samples from a single class or if we used all features, and note it as a leaf node

Top down approach: Grow the tree from root node to leaf nodes.

*ID3 (Iterative Dichotomiser 3)

Decision Trees: Package Delivery Prediction

- Given this dataset, let's predict package on time delivery (yes/no) using a Decision Tree.
- Iteratively split the dataset into subsets (branches), such that the final subsets (leaves) contain mostly one class.

3 features

Weather	Demand	Address	ontime
Sunny	High	Correct	No
Sunny	High	Misspelled	No
Overcast	High	Correct	Yes
Rainy	High	Correct	Yes
Rainy	Normal	Correct	Yes
Rainy	Normal	Misspelled	No
Overcast	Normal	Correct	Yes
Sunny	High	Correct	No
Sunny	Normal	Correct	Yes
Rainy	Normal	Misspelled	Yes
Sunny	Normal	Misspelled	Yes
Overcast	High	Misspelled	Yes
Overcast	Normal	Correct	Yes
Rainy	High	Misspelled	No

Best Feature to Split with?

[9+, 5-]

Class: Yes, No

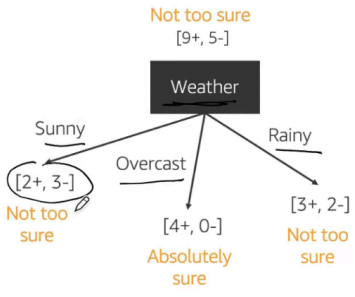
What feature ('Weather', 'Demand' or 'Address') to use to split the dataset, to best separate class 'No' from class 'Yes'?

[select the splits such that the descendent subsets are "purer" than their parents]

	Weather	Demand	Address	ontime
-1	Sunny	High	Correct	No
-2	Sunny	High	Misspelled	No
1	Overcast	High	Correct	Yes
2	Rainy	High	Correct	Yes
3	Rainy	Normal	Correct	Yes
-3	Rainy	Normal	Misspelled	No
4	Overcast	Normal	Correct	Yes
-4	Sunny	High	Correct	No
5	Sunny	Normal	Correct	Yes
6	Rainy	Normal	Misspelled	Yes
7	Sunny	Normal	Misspelled	Yes
8	Overcast	High	Misspelled	Yes
9	Overcast	Normal	Correct	Yes
-5	Rainy	High	Misspelled	No

Best Feature to Split with?

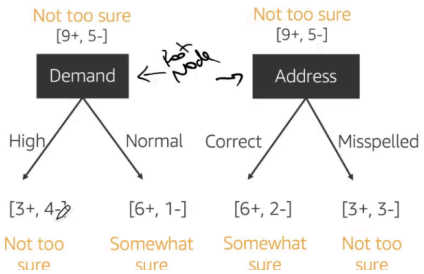
A good split results in overall less uncertainty (impurity). For example:



Weather	Demand	Address	ontime
Sunny	High	Correct	No
Sunny	High	Misspelled	No
Overcast	High	Correct	Yes
Rainy	High	Correct	Yes
Rainy	Normal	Correct	Yes
Rainy	Normal	Misspelled	No
Overcast	Normal	Correct	Yes
Sunny	High	Correct	No
Sunny	Normal	Correct	Yes
Rainy	Normal	Misspelled	Yes
Sunny	Normal	Misspelled	Yes
Overcast	High	Misspelled	Yes
Overcast	Normal	Correct	Yes
Rainy	High	Misspelled	No

Best Feature to Split with?

A good split results in overall less uncertainty (impurity). For example:



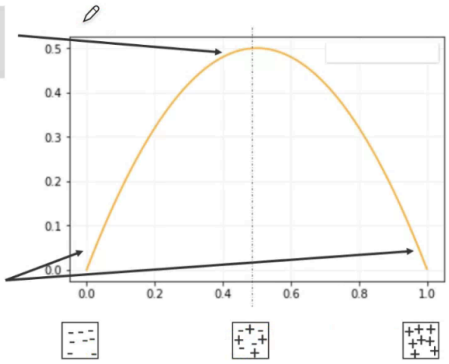
Weather	Demand	Address	ontime
Sunny	High	Correct	No
Sunny	High	Misspelled	No
Overcast	High	Correct	Yes
Rainy	High	Correct	Yes
Rainy	Normal	Correct	Yes
Rainy	Normal	Misspelled	No
Overcast	Normal	Correct	Yes
Sunny	High	Correct	No
Sunny	Normal	Correct	Yes
Rainy	Normal	Misspelled	Yes
Sunny	Normal	Misspelled	Yes
Overcast	High	Misspelled	Yes
Overcast	Normal	Correct	Yes

Rainy	High	Misspelled	No

How to Measure Uncertainty?

If we have mix of + and - samples:
High uncertainty

If we have only + samples or only – samples:
Low uncertainty



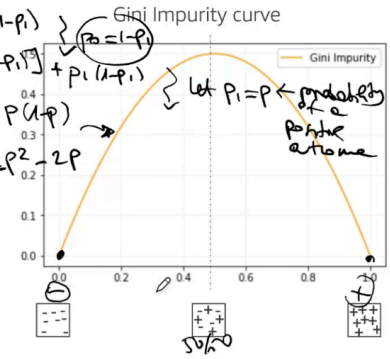
Measure Uncertainty

We will use Gini impurity:

$$i(p_1, ..., p_k) = \sum_{k=1}^{n} p_k(1 - p_k)$$

(n: number of classes, p_k : prob. of picking a datapoint from class k)

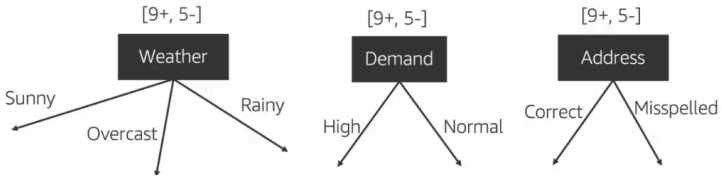
Another measure:
Entropy: More details



Information Gain & Feature Selection

Information Gain: Expected reduction in uncertainty due to selected feature.

$$\text{Gain} = \underbrace{i(p_1, ..., p_k)_{before}}_{\text{Impurity before split}} - \underbrace{i(p_1, ..., p_k)_{after}}_{\text{Impurity after split}}$$



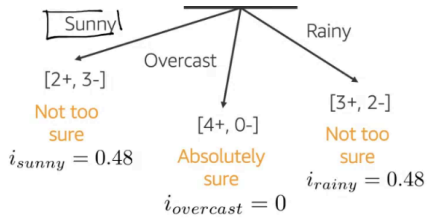
"Weather", "Demand" or "Address", which one should we select as feature to split?

Calculating Gini Impurity

$i_{before} = 0.46$
Not too sure
[9+, 5-]
Weather

Gini impurity:

$$i(p_1, ..., p_k) = \sum_{k=1}^{n} p_k(1 - p_k)$$



$$i_{sunny} = \frac{2}{5} * \frac{2}{5} + \frac{3}{5} * \frac{3}{5} = 0.48$$

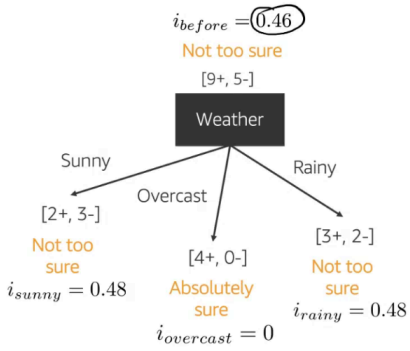
$$i_{overcast} = \frac{4}{4} * \frac{0}{4} + \frac{0}{4} * \frac{4}{4} = 0$$

$$i_{rainy} = \frac{3}{5} * \frac{2}{5} + \frac{2}{5} * \frac{3}{5} = 0.48$$

$$i_{weather} = \frac{5}{14} * i_{sunny} + \frac{4}{14} * i_{overcast} + \frac{5}{14} * i_{rainy}$$

$= 0.34$ (Weighted sum of impurities)

Information Gain & Feature Selection

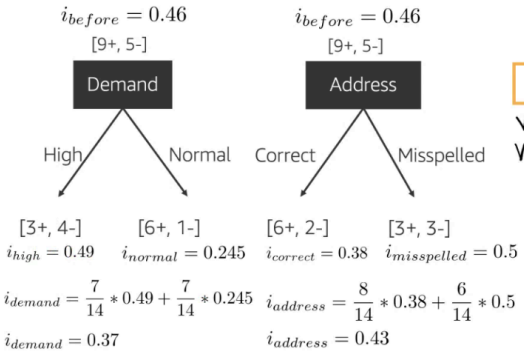


$$\text{Gain} = \underbrace{i(p_1, ..., p_k)_{before}}_{\text{Impurity before split}} - \underbrace{i(p_1, ..., p_k)_{after}}_{\text{Impurity after split}}$$

$i_{before} = 0.46$ $\quad$ $i_{weather} = 0.34$

Gain("Weather") = 0.46-0.34=0.12

Information Gain & Feature Selection



$$i_{demand} = \frac{7}{14} * 0.49 + \frac{7}{14} * 0.245$$

$$i_{demand} = 0.37$$

$$i_{address} = \frac{8}{14} * 0.38 + \frac{6}{14} * 0.5$$

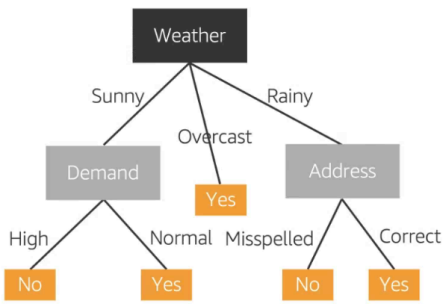
$$i_{address} = 0.43$$

Comparing gains for each feature:

Gain("Weather") = 0.46-0.34 = 0.12
Gain("Demand") = 0.46-0.37 = 0.09
Gain("Address") = 0.46-0.43 = 0.03

"Weather" has the highest gain of all, so we start the tree with the "Weather" feature as the root node!

Recap ID3 Algorithm



ID3 Algorithm*: Repeat

1. **Select** "the best feature" to split using Information Gain.
2. **Separate** the training samples according to the selected feature.
3. **Stop** if have samples from a single class or if all features used, and note a leaf node.
4. **Assign** the leaf node the majority class of the samples in it.

**To build a Decision Tree Regressor: 1. Replace the Information Gain with Standard Deviation Reduction. 3. Stop when numerical values are homogeneous (standard deviation is zero) or if used all features, and note it as a leaf node. 4. Assign the leaf node the average value of its samples.*

Decision Trees: Summary - Pros

- Simple to understand and easy to interpret
- Computationally efficient
  - Less frequent data lookup at training
  - A model is learned from the training data and used for inference
- Require very little data preprocessing
  - Some implementations handle both numerical and categorical data
  - No need for feature scaling; not affected by outliers
  - Do not require any assumptions on data distribution
  - Useful when data may be mixed, noisy, unusual distributed
- Highlight relevant features (feature selection)

Decision Trees: Summary - Cons

- Imbalanced datasets
  - Issue: Create biased trees if some classes dominate
  - Solution: Balance the data set
- Overfitting
  - Issue: Over-complex trees do not generalize well
  - Solution: Prune the trees to reduce complexity
- Unstable
  - Issue: Small variations in the training data might result in a completely different tree being generated
  - Solution: Build multiple trees where the samples and features are randomly sampled with replacement (ensemble methods – **bagging**)

Ensemble Methods: Bagging

Ensemble Learning

Ensemble methods create a strong model by combining the predictions of multiple weak models (aka **weak learners** or **base estimators**) built with a given dataset and a given learning algorithm.

We discuss Bagging and Boosting ensemble models.

Hyperparameter Tuning

Hyperparameter Tuning

- Hyperparameters are predefined parameters that affect the performance and structure of the ML algorithm.

  For example:

  - K Nearest Neighbors: n_neighbors, metric, …
  - Decision trees: max_depth, min_samples_leaf, class_weight, criterion, …
  - Random Forest: n_estimators, max_samples, …
  - Ensemble Bagging: base_estimator, n_estimators, …

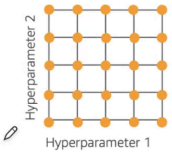
- Hyperparameter tuning looks for the best combination of hyperparameters (combination that maximizes model performance).

Hay 3 técnicas principales con sklearn

Grid Search in sklearn

1. GridSearchCV: sklearn basic hyperparameter tuning method, finds the optimum combination of hyperparameters by exhaustive search over specified parameter values - .fit(), .predict()

GridSearchCV(*estimator*, *param_grid*, *scoring=None*)



Example: Hyperparameters for a Decision Tree:

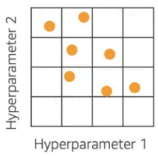
param_grid ={max_depth: [5, 10, 50, 100, 250],
min_samples_leaf: [15, 20, 25, 30, 35]}

Total hyperparameters combinations
5 x 5 = 25
[5, 15], [5, 20], [5, 25], [10, 15], …

Randomized Search in sklearn

2. RandomizedSearchCV: randomized search on hyperparameters
   - Chooses a fixed number (given by parameter *n_iter*) of random combinations of hyperparameter values and only tries those.
   - Can sample from distributions (sampling with replacement is used), if at least one parameter is given as a distribution.

RandomizedSearchCV(*estimator*, *param_distributions*,
n_iter=10, *scoring=None*)



Example: Hyperparameters for a Decision Tree:

param_grid ={max_depth: [5, 10, 50, 100, 250],
min_samples_leaf: uniform(15,35,5)}

Bayesian Search

3. Bayesian Search method keeps track of previous hyperparameter evaluations and builds a probabilistic model.
   - Balance exploration (exploring the whole sample space) and exploitation (exploiting promising areas)

- AWS SageMaker uses Bayesian Search for hyperparameter optimization.

Putting it all together

- In this notebook, we continue to work with our review dataset to predict the **target_label** field
- The notebook covers the following tasks:
  - Exploratory Data Analysis
  - Splitting dataset into training and test sets
  - Categoricals encoding and text vectorization
  - Train a Decision Tree Classifier, and Hyperparameter Tuning
  - Check the performance metrics on test set

Amazon SageMaker

MLA-TAB-DAY2-TREE-MODELS-NB.ipynb